ABRA — The Special Cut

The whole story, front to back: cheat card, the map of the models, then executive summary → deck → white papers → coach spec.

Will Hooper · 2026-07-22

A single consecutive read of everything ABRA. Start at the cheat card, glance at the interaction chart, then read straight through. Updated in the same pass as the code it describes.

Contents

PART 1

ABRA — the cast, on one card

Version 1.0 · 2026-07-22 · Will Hooper

> *Every model is a Pokémon. The Pokémon's **speed** tells you the model's **cost**: fast Pokémon = cheap/fast model, slow Pokémon = expensive/deep model. Every name is an acronym.*

What each one does

Model	Pokémon (speed)	Acronym	In one line	Cost	Status
ABRA	Abra	**A**utomated **B**attle **R**eplay **A**nalyzer	Collects live ladder games, stores them raw forever, models them. The platform everything sits on.	—	Built
JOLTEON	Jolteon (fastest)	**J**oint **O**dds, **L**adder-**T**rained **E**xpected-**O**utcome **N**etwork	Instant win probability between two teams, before any move.	⚡ microseconds	Built

Model	Pokémon (speed)	Acronym	In one line	Cost	Status
			~55% vs 49% coin flip.		
MEDICHAM	Medicham (medium)	**M**atchup **E**valuation, **D**amage-**I**nformed **C**HOMP-**H**euristic **A**pproximate **M**oves	Plays a matchup out a few turns with CHOMP's exact damage; averages many rollouts.	▯ milliseconds	Built
SLOWKING	Slowking (slow, wise)	**S**earch over **L**earned **O**pponent-belief **W**orld, **K**nowledge-**I**ntensive **N**ash **G**ame-solver	Deep, equilibrium-aware search over what the opponent might be hiding.	▯ seconds	Scaffold (research)
DITTO	Ditto	**D**ouble-oracle **I**terative **T**eam-**T**uning **O**ptimiser	Builds teams that beat the live meta; guarantees an answer to common	uses JOLTEON	Built

Model	Pokémon (speed)	Acronym	In one line	Cost	Status
			threats, ignores rare ones.		
KADABRA	Kadabra	**K**ey **A**nalysis of **D**ecisions, **A**dvice & **B**etter **R**eplay **A**nnotation	Turn-by-turn coach: rebuilds each scene, flags high/low rolls, names the better play.	uses CHOMP+ABRA	v1 built
CHOMP	Garchomp	(the bring-4 / lead-2 engine)	The Showdown userscript you actually play with: picks your 4 and your lead by exact damage.	—	Built (separate)

The supporting data models

Piece	What it produces
durable-ingest	The store: every game's teams, brings, leads, revealed

Piece	What it produces
	sets, result, ratings, bot flag, **and a per-turn stream** (move order → speed, damage % per move). Raw logs archived → any new field is a re-parse, never a re-pull.
dynamics	Observed **speed** (who-moves-first, Choice-Scarf hints) for 186 species, and observed **damage** distributions for 1,170 (attacker, move) pairs.
analyze (meta-usage)	Team % / bring % / lead % / win % per species at any rating cutoff. What CHOMP reads.

The one honest lesson

DITTO can **Goodhart** JOLTEON — build a team JOLTEON scores at 90% that is really ~12%. MEDICHAM's grounded rollouts catch it. That's why there's a fast guesser *and* a slower checker: **Tier-1 proposes, Tier-2 vets.** Their disagreement flags both hype (JOLTEON-high, MEDICHAM-low) and hidden gems (JOLTEON-low, MEDICHAM-high).

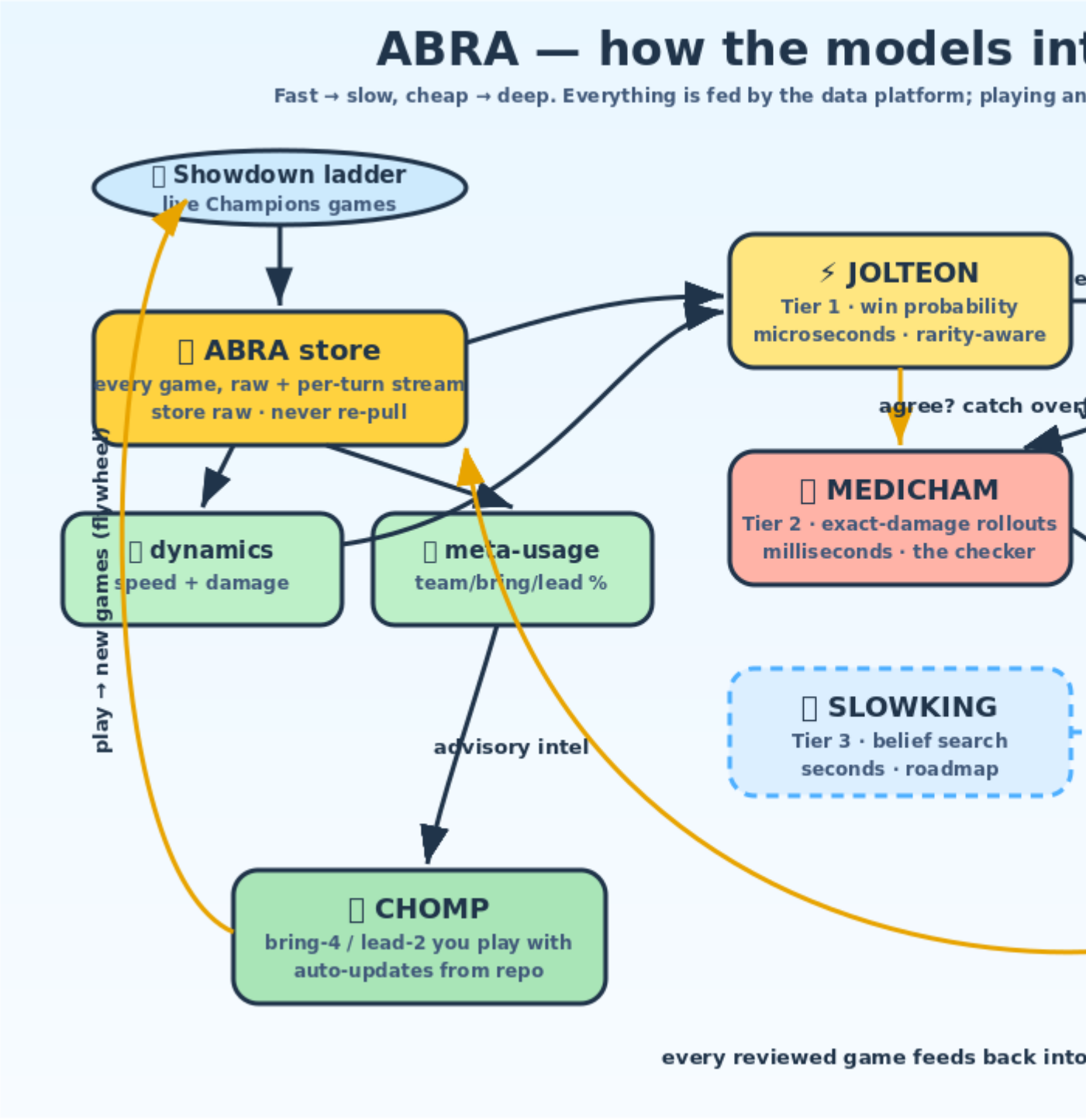
Who feeds whom (quick read)

- **ABRA → everything.** The store + dynamics + usage feed all models.
- **JOLTEON → DITTO.** Fast evaluator for the team search.
- **MEDICHAM → DITTO, KADABRA.** Grounded rollouts vet teams and answer "what if".
- **SLOWKING → KADABRA.** Deep lines (roadmap).
- **ABRA → CHOMP.** Usage as advisory intel; CHOMP auto-updates, so a smarter ABRA reaches your plugin with no reinstall.
- **KADABRA → ABRA.** Every reviewed game is appended back to the store (the flywheel from the coaching seat).

See the interaction chart: `architecture-diagram.svg`.

PART 2

How the models interact



PART 3

ABRA — Executive Summary

Version 1.1 · Last updated 2026-07-22 · Will Hooper

In one line

ABRA is the live-data platform for competitive Pokémon Champions: it continuously collects real ladder games and turns them into a durable dataset that feeds models of games and teams. The CHOMP bring-4 engine is one small, early consumer — proof the pipe delivers, not the purpose.

The problem

Choosing four of your six at team preview depends on what the opponent will bring. Public usage sites are stale and averaged; damage calculators see one matchup at a time. Nothing turns the *live* ladder into a model a decision engine can read and that improves as people play.

What ABRA does

- **Ingests the ladder automatically.** Public replays, pulled in parallel at ~200 games/second; the ~5,000 recent public games ingest in under a minute. A scheduled cloud job keeps it current hourly, with no operator.
- **Stores raw, analyses on top.** Every game is saved with every fact — both teams, brings, leads, observed sets, result, both ratings, and a bot flag. Any question (high-ladder only, humans only, just my games) is a filter on that store, **never** a re-download. This is the property that means the model can change forever without re-pulling.
- **Produces a usage model** — team%, bring%, lead%, win% per species, at any rating cutoff — and writes it where CHOMP reads it.

How it makes CHOMP smarter

CHOMP and ABRA are separate but connected. ABRA feeds CHOMP the ladder usage model as **advisory intel** — what the opponent will probably bring and lead. The pick itself covers the opponent's *whole six*: you cannot know which four they will bring, so CHOMP chooses the four that answer all six, and ABRA's prediction informs the read rather than gambling the bet. Because CHOMP auto-updates from its repository, a better ABRA model reaches the live plugin with no plugin change — the "gets smarter over time" loop, closed and tested.

Evidence

- Validated on **1,501 real ladder games** (2,292 human teams). The high-ladder cut (humans, 1300+) reveals distinct signal — top threats near 50% usage, win rates that separate (Kingambit 62%, Incineroar 65%).
- The top of the model independently reproduces a hand-curated threat list built from a **separate ~141,000-battle dataset** — external corroboration that the pipeline is correct.
- The ABRA→CHOMP flow-back is pinned by an automated test on a real battle: CHOMP loads the model and focuses the pick on the opponent's likely four against the user's six.

Honest limits

Revealed sets are partial (a Pokémon that never attacks reveals no moves). The model is descriptive, not a win-probability oracle. Bot and low-ladder games differ from strong play — which is exactly why they are tagged, so the consumer chooses the population.

The model family (the named cast)

The simulator is decomposed into tiers, each a Pokémon whose *speed* signals

the model's cost:

- **JOLTEON** — *Joint Odds, Ladder-Trained Expected-Outcome Network* (Tier 1, fastest): instant team-vs-team win probability. ~55% held-out vs ~49% coin flip; rarity-aware so rare picks aren't overrated. *Built.*
- **MEDICHAM** — *Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves* (Tier 2): short rollouts over CHOMP's exact damage. Rain beats sun 0.60, mirror 0.51. *Built.*
- **SLOWKING** — *Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver* (Tier 3, slowest/wisest): deep belief-search. *Scaffolded; honest research roadmap.*
- **DITTO** — *Double-oracle Iterative Team-Tuning Optimiser*: builds teams vs the live meta, usage-weighted so you get an answer to Basculegion, not Camerupt. *Built.*
- **KADABRA** — *Key Analysis of Decisions, Advice & Better Replay Annotation*: turn-by-turn coach that reconstructs each scene and flags high/low rolls. *v1 built.*
- **CHOMP** — the bring-4 / lead-2 engine you play with. *Built, separate-but-connected.*

Data now captured per game

Beyond teams/brings/leads, the extractor now records a **per-turn stream** — move order (→ real speed, Choice-Scarf detection) and exact damage % per move. 30,611 turns, 55,336 damaging hits. Raw logs are archived, so any *new* field is a re-parse, never a re-pull (proven: format tags backfilled onto 4,999 games with zero downloads).

The flywheel (the reason ABRA exists)

1. **Collect** live ladder games — *built and running*.
2. **Simulate** — game models from the data (JOLTEON, MEDICHAM) — *v1 built*.

3. **Optimise teams** against real meta threats (DITTO), iterating on itself — *v1 built*.
4. **Play** the best teams; CHOMP handles bring-4 / lead-2 — *built*.
5. **Feed back** — every enemy team you face (incl. every game KADABRA reviews) is auto-added.

The loop closes: more games → better simulator → better teams → more wins → more games. The most important early result is an *honest* one: DITTO can Goodhart JOLTEON (a "90%" team is really ~12%), which MEDICHAM catches — the tiered design earning its keep.

Status

v1.1 — the data platform is built and validated; Tiers 1–2 and the DITTO optimiser are built and measured; Tier 3 (SLOWKING) is scaffolded research. An interactive site (ABRA WORLD) runs JOLTEON live in the browser. CHOMP proves the pipe delivers.

Read next: deck · white paper · simulator/ML white paper · KADABRA spec

PART 4

ABRA — the deck

The Pokémon that reads the whole ladder

Version 1.1 · Last updated 2026-07-22 Will Hooper

> *Plain words only. The math and sources live in the white paper.*

Slide 1 — What ABRA is

A program that watches thousands of real Pokémon games and learns what everyone is playing.

It reads public battle replays from Pokémon Showdown — the Champions Reg M-B ladder — and turns them into a live picture of the metagame. Then it hands that picture to CHOMP.

Slide 2 — CHOMP and ABRA are a team

- **CHOMP** picks your four and your lead in the 30 seconds before a battle.
- **ABRA** tells CHOMP what the ladder actually brings, so those picks are based on reality.

Two separate programs, one loop. CHOMP is the hands; ABRA is the eyes.

Slide 3 — Why this matters

To choose your four well, you need to know what your opponent will bring. A

static usage list from a website is weeks old and averaged over everyone. ABRA reads the *current* ladder and updates itself.

Slide 4 — It reads thousands of games, fast

The replay site lets anyone download public games. ABRA grabs them in parallel — about **200 games a second** — so the last few thousand games ingest in under a minute.

There are roughly **5,000** recent public games available at any moment, and new ones upload constantly.

Slide 5 — The one rule that saves us from ever redoing it

Store everything, decide later.

For every game, ABRA saves *all* the facts — both teams, what each side actually brought, what they led, the moves and items we saw, who won, both players' ratings, and whether a player was a bot.

Because everything is stored, any question — "show me only strong players," "hide the bots," "just my games" — is answered instantly from the saved data. **We never have to download the games again.**

Slide 6 — Bots and rating

The ladder has automated **bot** accounts mixed in with real people. ABRA flags them. It also saves each player's rating. So you can look at the *whole* ladder, or *only humans above 1300*, or anything in between — your choice, no re-downloading.

That filter matters: at high ladder the picture sharpens — the top threats show up on half of all teams, and some win far more than others.

Slide 7 — Keeping your games separate

Your own games are just a filter: any game with your Showdown name. So ABRA can show you the whole ladder **and** your personal record from the same pile of data.

Changed your Showdown name? Add the new name to a short list once. Nothing is lost, nothing re-pulled. No accounts, ever.

Slide 8 — It works for anyone

The ladder picture belongs to everyone. Your personal view is just your name. So a friend points ABRA at their own name and gets their own analysis from the same shared data. One tool, many users, no sign-ups.

Slide 9 — It runs itself

A scheduled job in the cloud wakes up every hour, grabs the newest games, updates the model, and saves it. You don't run anything. The picture stays current on its own — and because CHOMP updates itself too, your live plugin quietly gets smarter while you play.

Slide 10 — Now it reads *inside* the games too

ABRA no longer just notes who was on each team. For every game it now records, turn by turn, **who moved first** (that reveals real speed — even a hidden Choice Scarf) and **exactly how much damage each move did**.

Thousands of games of this add up to a real picture of the physics: we know Basculegion's Wave Crash takes about 62% on average, and we can tell when someone got a lucky roll.

Slide 11 — The cast of models

Each part of the plan is a Pokémon, and the Pokémon's **speed tells you how expensive the model is**:

- ⚡ **JOLTEON** (fastest) — instant "who's favoured?" between two teams. Built.
- **MEDICHAM** (medium) — actually plays the matchup out a few turns with real damage math. Built.
- **SLOWKING** (slow, wise) — the deep thinker that plans around what the opponent is hiding. Still being built.
- **DITTO** — builds teams that beat the live ladder, making sure you answer the *common* threats. Built.
- **KADABRA** — the coach that replays your game with you and points out the better move. Built.
- **CHOMP** — the engine you actually play with, picking your 4 and your lead.

Slide 12 — The most honest result we have

When DITTO built a team to beat JOLTEON's scoring, JOLTEON said it would win **90%** of the time. Then MEDICHAM played it out for real and said **12%**. JOLTEON had been fooled — it over-trusted a few rare Pokémon. That's not a bug we hid; it's *why* we have a fast guesser and a slower checker. The disagreement between them is a feature: it flags both hype and hidden gems.

Slide 13 — ABRA WORLD (the fun part)

There's a little snowy town website where every model is its own building. You can pick two teams from sprites and watch JOLTEON — the *real* model — call the odds right in your browser, with confetti when the battle resolves. It's `web/index.html`.

Slide 14 — Honest limits

- A Pokémon that never attacked in a replay tells us nothing about its moves — so what we know about each set is partial.
- These are **descriptions** of what people play, not a prediction of who wins.
- Low-ladder and bot games look different from strong play — which is exactly why we tag them and let you choose.

Slide 15 — Read more

The white paper — the data model, the ingest, the math, the sources: **ABRA-whitepaper.md**

The simulator / ML white paper — the machine learning behind the model family: **ABRA-simulator-whitepaper.md**

The coach spec — how KADABRA teaches: **KADABRA-coach-spec.md**

The changelog — what changed and why: **CHANGELOG.md**

PART 5

Modelling a Live Metagame from Public Replays

A technical description of ABRA, the Automated Battle Replay Analyzer

Version 1.0 · Last updated 2026-07-22 Will Hooper · ABRA

> *This is a living document, updated in the same pass as any change to the code. New information is appended; a prior conclusion is not silently rewritten. See `CHANGELOG.md`.*

Abstract

ABRA is a live-data platform for competitive Pokémon Champions (Reg M-B). It continuously ingests public battle replays from Pokémon Showdown, extracts the durable facts of every game, and stores them as a growing dataset — the foundation for **modelling games and teams**. The near-term model is a metagame usage model; the larger goal is a simulator that learns which teams and which in-game choices win. This paper states the data model, the ingest architecture, the first analysis, and the limits. The governing design rule — *store raw, analyse on top* — is defended as the property that lets every model built on the data change without ever re-fetching. A team-preview engine (CHOMP) is one small, early consumer of the output; it is a proof of delivery, not the purpose.

1. The problem

Competitive VGC has many hard questions — which six to build against the meta, which four to bring, how a matchup tends to play out. Answering any

of them well needs data on how the ladder *actually* plays, kept current as it changes. Existing tools give static usage snapshots (Pikalytics) or per-game damage math (calculators); none continuously collects live games into a durable dataset that downstream models — team optimisers, game simulators, decision engines — can all build on. ABRA is that collection-and-storage layer. The bring-4 engine (CHOMP) is the first small model on top.

2. The data source

Pokémon Showdown exposes public replays through a stable API:

- `search.json?format=<id>` — recent public replays for a format, paginated (~51/page, ~100 pages).
- `search.json?user=<name>` — a specific player's public replays.
- `<id>.log` — the full battle protocol for one replay.

Every replay is a public artifact already attached to public usernames; ABRA reads nothing private and creates no accounts (see `SECURITY.md`).

3. The record — what we store per game

The extractor (`engine/durable-ingest.js`, `extract()`) turns one `.log` into one durable record:

Field	Meaning
`id`, `date`	replay id and upload time
`p1`, `p2`	`{name, rating, bot}` per player — rating from the `|player|` line, `bot` by name pattern
`six.p1/p2`	the revealed team of six (`|poke|` lines at preview)
`brought.p1/p2`	the Pokémon actually sent out (`|switch|` lines)
`lead.p1/p2`	the first two sent out

Field	Meaning
`sets`	per species, the moves / item / ability the replay *revealed*
`winner`	the winning name (`|win|`)

This is deliberately richer than any single analysis needs. That richness is the insurance behind the design rule in §5.

4. Ingest architecture

- **Concurrent fetch.** A fixed-size promise pool fetches logs in parallel. Measured throughput is ~200 logs/sec, so the full ~5,000 recent public replays ingest in about 25–30 seconds.
- **Incremental and idempotent.** The store is append-only JSON Lines keyed by replay id. Each run reads existing ids, requests recent pages, and appends only games not already present. Re-running never duplicates and never re-fetches stored games.
- **Continuous collection.** A scheduled GitHub Action runs the ingest hourly and commits the refreshed store and model, so the metagame model tracks the live ladder with no operator.

5. The governing rule — store raw, analyse on top

Every analysis and every filter runs over the stored records, not over the network. Concretely:

- The rating cutoff, the humans-only filter, and the "just my games" filter are all computed from the same store at read time.
- Changing how games are *placed* (a different rating tier, excluding bots, a new archetype tag) is a re-computation, **never** a re-pull.

The alternative — deciding the population at fetch time — would force a re-fetch every time the question changed. Storing raw makes the fetch a one-time cost and the analysis free to evolve. This is the single most important property of the system.

6. The usage model

For a chosen population, `engine/analyze.js` computes per species:

```
teamRate  = teams containing the species        / teams
bringRate = games the species was brought        / games it was on the team
leadRate  = games the species was led            / games it was on the team
winRate   = games won when the species was brought / games it was brought
```

Species below a small sample floor (n ≥ 8) are dropped. The result is written to `data/meta-usage.json`, the model CHOMP reads.

6.1 A validated result

On 1,501 stored games (2,292 human teams), the raw ladder and the high-ladder cut (humans, rating ≥ 1300, 530 teams) differ meaningfully — the high-ladder cut lifts top-threat usage toward 50% and separates win rates (e.g. Kingambit 62%, Incineroar 65%). The top of the raw table independently reproduces a hand-curated threat list built from a separate ~141k-battle dataset (Basculegion, Whimsicott, Garchomp, Kingambit), which is external corroboration that the pipeline is correct.

7. Feeding CHOMP

CHOMP is a separate project — only the bring-4 / lead-2 engine. ABRA feeds it the ladder usage model (`meta-usage.json`) as **advisory intel**: what the opponent will *probably* bring and lead. CHOMP can sharpen that into a matchup-aware prediction — their best four against your six — but the **pick itself covers the opponent's whole six**. A player cannot know which four the opponent will bring, so CHOMP optimises for coverage over all six rather than gambling on a predicted four; ABRA's prediction informs the read, not the bet. Because CHOMP auto-updates from its repository, an improved ABRA model reaches the live plugin with no plugin change. This is the "gets smarter over time" loop, made concrete, and it is pinned by `CHOMP/tests/test-meta-flow.js`.

8. The flywheel — where this is going

ABRA's data platform is stage one of a self-improving loop. Each stage feeds the next, and the last stage feeds the first:

1. **Collect** — continuously ingest live ladder games into the durable store. *(built and running)*
2. **Simulate** — build a near-perfect game simulator from that data: given two teams, model how the matchup tends to play out. *(roadmap)*
3. **Optimise teams** — run hypothetical teams against the real metagame threats in the simulator and let the search iterate on itself, converging on sixes that beat what the ladder actually brings. *(roadmap; the meta threat model that feeds it is built)*
4. **Play** — take an optimised team onto the ladder; CHOMP handles the bring-4 / lead-2 in each game. *(CHOMP built)*
5. **Feed back** — every opponent team you face is auto-added to the store, so playing the game grows the dataset. *(the ingest + store support this directly; the auto-add-on-play hook is roadmap)*

The loop closes: more games → a better simulator → better teams → more wins → more games. The value is not any single model but the flywheel — the system gets stronger the more it is used. This is why the design rule (*store raw, analyse on top*, §5) matters so much: every model in the loop is built on the same growing dataset, and none of them ever forces a re-pull.

Honest status. Stage 1 is built and validated (§4, §6.1). Stages 2 and 3 — the simulator and the self-iterating team optimiser — are not yet built; they are the reason ABRA exists and the next work. CHOMP (stage 4) is live. The paper does not claim the flywheel is spinning yet; it claims the foundation that makes it possible is in place.

9. Limitations

1. **Revealed sets are partial.** A Pokémon that never attacked reveals no moves; items/abilities appear only when they trigger. Set inference is a

lower bound, not the full set.

2. **Descriptive, not predictive.** Usage and win rates describe the population; they are not a win-probability model.
3. **Population choice matters.** Bot and low-ladder games differ from high-ladder play. ABRA tags rating and bots so the consumer chooses the population; it does not decide for them.
4. **Store growth.** The append-only store grows without bound; pruning by date is future work.

10. References

1. Pokémon Showdown replay API — `replay.pokemonshowdown.com`.
2. `docs/reg-mb-threat-list.md` (CHOMP) — the independently curated threat list used for validation.
3. Keep a Changelog; Semantic Versioning; ASD-STE100; Diátaxis.

Companion documents. Slide deck · Technical documentation · Changelog

PART 6

Learning a Battle Simulator from Logged Replays

Machine-learning foundations for a tiered VGC game model

Version 1.1 · Last updated 2026-07-22 Will Hooper · ABRA

> ***Build status (v1.1):*** *Tiers 1 (JOLTEON) and 2 (MEDICHAM) and the DITTO outer loop are built, tested, and measured (see §12). The per-turn dynamics model is built. Tier 3 (SLOWKING) is scaffolded and honestly roadmap. Numbers below are from real runs.*
>
> *A research white paper on the modelling problem behind ABRA's stages 2–3 (the simulator and the team optimiser). It states the problem formally, surveys the relevant machine-learning literature, derives the math for each modelling tier, and gives an honest feasibility assessment. Living document; updated in the same pass as the code it describes.*

Abstract

ABRA collects a growing corpus of Pokémon Champions (Reg M-B) battle replays. We want to turn that corpus into a *simulator*: a model that, given two teams, predicts how their game tends to resolve, so that hypothetical teams can be optimised against the live metagame without playing thousands of real games. The underlying game — a two-player, zero-sum, simultaneous-move, stochastic game of imperfect information — is implemented by a closed engine we cannot query. We therefore treat the problem as **learning a model from logged data**. This paper formalises the game, decomposes the simulator into three modelling tiers of increasing fidelity and cost (an outcome model, a hybrid rollout model, and a learned dynamics model),

derives the estimator and its failure modes for each, frames team optimisation as the outer loop of a self-improving flywheel, and specifies an evaluation protocol that is honest about the irreducible variance of the domain. We ground each tier in the 2025 Pokémon-AI literature — PokéChamp, Metamon, and VGC-Bench — and conclude with a feasibility assessment: tier 1 is immediately achievable on ABRA's data, tier 2 reuses the existing CHOMP damage engine, and tier 3 is a genuine research effort that recent work shows to be possible but expensive.

---

1. Introduction

A competitive VGC player faces three nested questions: which six Pokémon to build, which four to bring, and how to play the resulting game. CHOMP answers the middle question with exact damage math. The hardest lever, and the one with the most upside, is the first: *which six beats the current metagame.* Answering it by trial on the ladder is slow and high-variance. The alternative is to **simulate** — to evaluate a candidate team against the meta in software, iterate, and only then play. This paper is about building that simulator from the data ABRA already collects.

The central obstacle is that the Champions battle engine is **closed**. Unlike the standard Smogon formats served by `pokemon-showdown`'s open-source simulator — which the poke-env library exposes to RL agents — the Champions ruleset and its stat system are not available as a queryable environment. We cannot roll the true dynamics forward for a hypothetical state. What we *do* have is a large and growing set of **observed trajectories**: real games, each a sequence of states and joint actions ending in a win or loss. Learning a usable model from such logs, with no ability to interact with the true environment, is exactly the setting of **offline (batch) reinforcement learning** and **model learning / system identification**. This is not a toy analogy: Metamon (§9) reconstructs 5M+ trajectories from human Showdown replays and trains competitive agents on them offline.

2. The game, formally

A single VGC battle is a **partially observable stochastic game (POSG)**, and specifically a two-player, zero-sum, *simultaneous-move* Markov game with imperfect information. We fix notation.

- **State** `s ∈ S`. The full state comprises both teams' species, sets (moves, item, ability, the Champions SP spread), current HP, status, stat stages, field conditions (weather, terrain, screens, Trick Room), and which Pokémon are active. Level 50; no Tera in Champions.
- **Players** `i ∈ {1, 2}`.
- **Actions**. At each turn both players choose *simultaneously* from a legal set `A_i(s)` — for each of two active Pokémon, a move (with a target) or a switch. The **joint action** is `a = (a_1, a_2)`. Team preview is a special first move: each player chooses an ordered bring of four from six, `A_i = {ordered 4-subsets of the six}`, `|A_i| = 6·5·4·3 = 360`.
- **Transition kernel** `T(s' | s, a)`. Stochastic: damage rolls (16 discrete outcomes, 85–100%), accuracy, secondary-effect procs, critical hits, speed ties. This kernel is the *engine*, and it is what a full simulator must reproduce.
- **Reward**. Zero-sum and terminal: `r = +1` for the winner, `-1` for the loser; `0` elsewhere. Define the **value** of a state to player 1 as `V(s) = E[r | s, play]` under a solution concept.
- **Observation**. Player `i` sees `o_i = O_i(s)`: their own team fully, and of the opponent only what has been revealed (the six species at preview; each move/item/ability only once used or triggered). This partial observability is first-class — it is why opponent modelling is a distinct problem, and why belief states matter.

Two structural facts shape everything downstream:

1. **Simultaneous moves ⇒ mixed strategies.** Because both players commit without seeing the other's choice, the per-turn subgame is a matrix game whose optimal solution is generally a *mixed* (randomised) Nash equilibrium, not a single best move. A simulator that assumes

deterministic opponent play is systematically wrong at exactly the decision points that matter.

2. **Imperfect information ⇒ belief states.** The relevant object is not `s` but a distribution over states consistent with the observation history — a belief `b(s)`. Solving the game exactly is the province of **counterfactual regret minimisation (CFR)** and its deep variants; approximations are unavoidable at VGC's scale.

The team-building question sits one level above the battle: it is a **Bayesian game** in which a player commits to a team before knowing the opponent's, and payoffs are the expected battle values integrated over the opponent distribution the metagame induces.

3. Three modelling tiers

A single "simulator" of full fidelity is neither necessary nor tractable for every use. We decompose it into three tiers, coarse-to-fine, and choose per use (broad search vs. finalist vetting). This is the standard model-based pattern: cheap approximate models for planning breadth, expensive accurate models for depth.

Tier	Object learned	Query	Cost	Primary use
1. JOLTEON (outcome)	`Pψ(win | team_A, team_B)`	one forward pass	~µs (fastest)	search over thousands of teams
2. MEDICHAM (hybrid rollout)	CHOMP damage + learned policies	a few short rollouts	~ms	ranking finalist teams

Tier	Object learned	Query	Cost	Primary use
3. SLOWKING (learned dynamics)	`Tθ(s' | s, a)` + value/policy	belief search / playout	~s (slowest)	deep vetting, a matchup

The rest of the paper derives each.

3.1 The model family, named

Each model gets its own name, in the CHOMP / ABRA tradition — and the Pokémon's *speed* signals the model's cost, fast to slow:

- **JOLTEON** — *Joint Odds, Ladder-Trained Expected-Outcome Network* (Tier 1). The **fastest** model (Jolteon is the quickest of the cast): an instant win-probability call between two teams, from Bradley–Terry strengths plus CHOMP-derived damage/coverage features (§4.3.1). One forward pass, microseconds — fast enough to score a whole team search.
- **MEDICHAM** — *Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves* (Tier 2). Middle of the pack, in name and in speed: it plays a matchup out a few turns using **CHOMP's exact damage engine** and behaviour-cloned move priors. Grounded, approximate, mid-cost.
- **SLOWKING** — *Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver* (Tier 3). The **slowest** and deepest: a slow but wise brain that searches over belief states on a learned dynamics model toward equilibrium play. Slow on purpose — you run it only on finalists.
- **DITTO** — *Double-oracle Iterative Team-Tuning Optimiser* (§8, the outer loop). It tries team after team against the meta and transforms toward the strongest six, the way Ditto tries every form.

CHOMP (the bring-4 engine) and the coach are *consumers* of these models, not models themselves. The naming makes the pipeline legible and the

speeds honest: ABRA feeds MEDICHAM and JOLTEON; those and SLOWKING feed DITTO; DITTO's teams and SLOWKING's lines reach the player through CHOMP and the coach; the games played feed ABRA. Fast Pokémon do the cheap, broad work; the slow one does the deep work.

4. Tier 1 — JOLTEON, the outcome model

4.1 Objective

Learn a function that maps an ordered pair of teams to a win probability: `Pψ(y = 1 | x_A, x_B)`, where `y = 1` iff team `A` wins and `x` is a feature encoding of a team. This is supervised binary classification on the log: each stored game contributes one labelled example (and its mirror, with the label flipped, to enforce the antisymmetry `P(A beats B) = 1 − P(B beats A)`).

4.2 The Bradley–Terry / Elo backbone

The natural, well-founded baseline treats each team (or, more usefully, each *component*) as having a latent strength and models paired-comparison outcomes with the **Bradley–Terry** model:

```
P(A beats B) = σ( β_A − β_B ),   σ(z) = 1 / (1 + e^{−z}).
```

Fitting `β` by maximum likelihood over the observed win/loss pairs is exactly logistic regression; online, the same model with a specific update rule is **Elo**. Two refinements matter for VGC:

- **Teams are compositional.** Rather than a strength per *team* (there are astronomically many), parameterise strength as a sum over the team's members and interactions: `β_A = Σ_{p∈A} w_p + Σ_{p,q∈A} u_{pq}`, learning a per-species main effect `w_p` and pairwise synergy `u_{pq}`. This is a factorisation-machine view and it generalises to unseen teams built from seen parts — the property we need, since we will score *hypothetical* teams.
- **Matchup structure (advantage cycles).** Pokémon has rock–paper–scissors matchups: strength is not fully captured by a single scalar. The

Blade–Chest and low-rank bilinear extensions add a vector per team and score `β_A − β_B + ⟨c_A, m_B⟩ − ⟨c_B, m_A⟩`, which can represent non-transitive advantage (A beats B beats C beats A). Including this term is what lets the model express "this team preys on the current meta" rather than "this team is generically strong."

4.3 Features

`x` should be a fixed-length encoding of a team invariant to Pokémon order: a multi-hot species vector (dimension = legal roster), augmented with observed-set summaries (lead rate, common item, revealed moves aggregated from ABRA's `sets` field), archetype tags (weather, Trick Room, Tailwind), and speed-tier and type-coverage summary statistics. A neural encoder (a small permutation-invariant DeepSets/Transformer over the six member embeddings) is the expressive upgrade once the linear model saturates.

4.3.1 CHOMP's threat scoring as damage-grounded features

Species identity alone is a weak feature — two teams with the same six can play very differently, and what actually decides a matchup is *who KOs whom*. CHOMP already computes exactly this, and its output is the strongest feature JOLTEON can eat. CHOMP scores threats with the real Gen-9 damage pipeline on the Champions SP stat system and reports, for an attacker–defender pair, **pKO** — the fraction of the 16 damage rolls (85–100%) that knock out — combined with a speed check (does the attacker move first). Its bring-4 chooses the four that maximise KO-pressure/coverage across the opponent's six.

For JOLTEON, build the **coverage matrix** `K` where `K_{pq} = pKO(p → q)` over every attacker `p` in team A and defender `q` in team B (and its transpose for B→A), plus the speed-order mask. Summaries of `K` — how many of B's six A can OHKO, the best-case and worst-case coverage, symmetric speed control — become the features that carry most of JOLTEON's signal, because they are the real mechanics of the matchup rather than a usage proxy. This is grey-box modelling: CHOMP supplies the physics of a single exchange;

JOLTEON learns how those exchanges aggregate into a game outcome across thousands of real results. CHOMP's own validation (its white paper, validation report, and referee report) is therefore part of JOLTEON's foundation — the feature generator is already tested.

4.4 Estimation and calibration

Maximise the regularised conditional log-likelihood:

```
ψ* = argmax_ψ  Σ_g [ y_g log Pψ(x_{A,g}, x_{B,g}) + (1−y_g) log(1 − Pψ(·)) ] − λ‖ψ‖².
```

A win **probability** is only useful if it is *calibrated* — when the model says 70%, the team should win ~70% of the time. We measure calibration with the **Brier score** `(1/N) Σ (Pψ − y)²` and reliability diagrams, and correct residual miscalibration with **Platt scaling** or isotonic regression on a held-out split. Calibration, not raw accuracy, is what the team optimiser consumes, because it integrates these probabilities over the meta.

4.5 What tier 1 can and cannot do

It answers "does team A tend to beat team B?" from data, generalises to unseen teams via compositional features, and runs fast enough to score a whole search. It does **not** know *why* — it has no notion of turns, and it cannot tell you the line of play. It is a value function without a model. That is exactly enough for the outer optimisation loop and nothing more.

5. Tier 3 — SLOWKING, learning the dynamics (the true "reconstruction")

Tier 3 is what "reconstruct the simulator" literally means: learn the transition kernel `Tθ(s' | s, a)` and a policy/value on top, from logged transitions.

5.1 The dynamics model

Each replay is a sequence `(s_0, a_0, s_1, a_1, …, s_H, r)`. Every consecutive triple `(s_t, a_t, s_{t+1})` is a supervised example for a **world model**. Fit by

maximum likelihood:

```
θ* = argmax_θ  Σ_{t}  log Tθ(s_{t+1} | s_t, a_t).
```

Because the true kernel factorises (each active Pokémon's move resolves largely independently given speed order), a structured model predicting damage-roll distributions, status procs, and stat changes per move is far more sample-efficient than a monolithic next-state predictor. Damage is *already* known in closed form (CHOMP's engine); the learned part is the residual — accuracy, secondary effects, switching outcomes, and the opponent's hidden set. This is a **grey-box** system identification problem: known physics plus a learned residual, which needs far less data than a black-box learner.

5.2 Partial observability

We never observe `s`; we observe `o` and must maintain a **belief** `b_t = Pr(s_t | o_{≤t}, a_{<t})`. The opponent's hidden set is the main latent variable. ABRA's usage model supplies the **prior** `Pr(set | species)` (how the ladder builds each Pokémon), and each revealed move/item is a Bayesian update. A particle filter over sampled opponent sets, each rolled forward through `Tθ`, is the standard tractable representation. Note the clean division of labour: ABRA (data) supplies the prior; the dynamics model supplies the update.

5.3 Solving the game on the learned model — search over beliefs, not states

With `Tθ` and beliefs, the game must be searched. Because the information is imperfect, the correct object is **not** a tree over states but a search over **public belief states (PBS)** — distributions over what each side could hold given everything commonly observed. This is the central idea of **ReBeL** (Recursive Belief-based Learning), which combines deep RL with search over PBSs and *provably converges to a Nash equilibrium* in two-player zero-sum imperfect-information games; in the perfect-information limit it reduces to AlphaZero. **Player of Games** and **Student of Games** later unified perfect- and imperfect-information search into one algorithm. This family — RL +

search over beliefs — is the principled target for a Pokémon solver, and it subsumes the two simpler options: a per-turn **matrix-game Nash** (a small linear program at each node) with **minimax/expectimax** over joint actions, which is exactly PokéChamp's architecture (LLM samples actions, models the opponent, estimates leaf values; Elo 1300–1500); and the search-free route of a **policy/value network learned offline**, as Metamon does via behavioural cloning plus offline RL and AlphaStar Unplugged does at scale.

5.4 Why offline RL is hard here (and how the literature copes)

Learning a policy or value purely from logs suffers **distributional shift**: the model is queried at state–action pairs the logging players never took, where its estimates are unconstrained and optimistic. The canonical fixes are **conservatism** — Conservative Q-Learning (CQL) penalises out-of-distribution action values; Implicit Q-Learning (IQL) avoids querying unseen actions entirely — and **return-conditioned supervised learning** — the Decision Transformer casts offline RL as sequence modelling, which is the family Metamon uses. Off-policy **evaluation** (weighted importance sampling, doubly-robust estimators) lets us score a policy from logs without deploying it, with variance that grows with policy divergence. The blunt honest point: without environment access, every tier-3 estimate is an extrapolation, and its trustworthiness must be *measured*, not assumed.

5.5 You do not simulate every path — the policy prunes the tree

The intuition that "*you would not simulate all the paths, machine learning just chooses the few viable ones*" is exactly how modern game AI is made tractable, and it has a precise form. The branching factor of a turn is large (each of two actives can move-with-target or switch), and the tree explodes geometrically with depth. AlphaZero tames this with a **policy prior** that biases the search toward promising actions; for genuinely large action spaces the fix is to **sample** rather than enumerate: **Sampled MuZero** expands only a subset of actions drawn from the prior, **AlphaStar** restricts StarCraft's enormous action space to a handful of policy-sampled actions per node, and

Gumbel MuZero makes this sampling near-optimal with few simulations. PokéChamp is the Pokémon instance: the LLM proposes only the plausible moves, so the minimax tree never touches the overwhelming majority of legal-but-pointless lines.

For ABRA this means the learned policy is not a luxury on top of the simulator — it *is* what makes the simulator usable. A well-trained policy assigns almost all probability to the two or three viable options in a position, so search explores a tree with an effective branching factor of a few, not a few dozen. The cost of a "near-perfect" playout is therefore governed by the *policy's* quality, not by the raw size of the game tree. This also answers a practical worry: we never need to model every interaction explicitly by hand — a policy trained on enough real games learns which interactions matter and silently prunes the rest.

6. One model, many queries — playing and testing teams from a shared core

Is the team-tester the same model as the game-simulator? They should share a core. The AlphaZero / MuZero pattern is a single network with a shared representation and two heads — a **value** head `Vφ(belief)` and a **policy** head `πφ(action | belief)`. That one core answers both of ABRA's questions:

- **Testing teams against the meta** is the value head, marginalised over play and over the meta distribution: `E_{o~D}[ Vφ(team, o) ]` (§7). No game needs to be rolled out turn-by-turn to *rank* teams if the value head is calibrated — this is precisely tier 1, and it can be **distilled** from the full model into a fast outcome head so that team search stays cheap.
- **Giving the player the line** is the policy head plus search: from the current belief, the model returns the best move(s) — the very thing the coach surfaces. Because the model has learned how the mechanics and interactions resolve, it can hand the player a *recommended path*, not just a verdict. Every uploaded game both trains this core and is a state it can now advise on.

So the honest architecture is **one learned world/value/policy core, queried three ways** — as a fast team-evaluator (distilled value head), as a play-advisor (policy + belief search), and as the coach's explanation layer. Whether tier 1 remains a separate lightweight model or becomes a distilled head of the tier-3 core is an engineering choice, not a conceptual one; both are the same object viewed at different cost.

6.1 Continual learning — the model updates with every upload

"With every game upload the model improves" is **continual (online) learning**, and it has a known failure mode — **catastrophic forgetting**, where new data overwrites old competence. The standard guards are a **replay buffer** (keep training on a mix of old and new games, never new alone), periodic re-fitting rather than pure online updates, and continuing to ingest the *whole* public ladder so the model does not narrow to only the games the current policy likes. Under these guards the flywheel (§8) is a continual-learning loop whose data distribution is steered, deliberately, toward the positions the player actually reaches.

7. Tier 2 — MEDICHAM, the pragmatic middle

Status: built (`engine/medicham.js`, `tests/test-medicham.js`). Rain core beats sun core 0.60, mirror 0.51, ~1s / 300 playouts; v1 is sequential-singles (honest scope, true doubles is roadmap).

MEDICHAM sidesteps learning dynamics by *specifying* them: it uses CHOMP's exact damage engine — the same pKO-over-16-rolls threat scoring as tier 1 — as a hand-built `T` for the dominant effect (damage), pair it with cheap policies (best-damage heuristics, or behaviour-cloned move priors from ABRA's `sets`), and roll a matchup out a few turns with a handful of stochastic samples to average over damage rolls. It is grounded in real math, requires no model training, and reuses code that already exists and is already tested. It is the right first *playout* model precisely because its errors are known and bounded (it ignores multi-turn tactics), and it is a strong baseline

against which any learned tier-3 model must justify its cost.

8. DITTO — team optimisation, the outer loop

Status: built (`engine/ditto.py`). Uses JOLTEON as evaluator against a gauntlet of real ladder teams, double-oracle rounds, usage-weighted threat coverage, and a bias report. Its headline finding is a *negative* one, stated plainly in §12: optimising against JOLTEON Goodharts it (a "90%" team is really ~12% by MEDICHAM), which is exactly why finalists are vetted by Tier 2. Rarity shrinkage (§4.4) and the proven-meta pool are the guards; MEDICHAM is the check.

Given any tier as an evaluator `Ê[win | team]`, optimise the team. Formally, choosing a team `t` to maximise expected win rate against the **meta distribution** `D` (which ABRA measures):

```
t* = argmax_{t ∈ legal}  E_{o ~ D} [ Pwin(t, o) ].
```

The legal team space is astronomically large (hundreds of species choose six, times sets), so exhaustive search is out. The tractable families:

- **Evolutionary / local search.** Seed from meta archetypes or a user pool, mutate one slot at a time, keep improvements. Cheap, anytime, and matches how humans iterate.
- **Bayesian optimisation / bandits.** Treat team evaluation as expensive-noisy; use a surrogate over the team-feature space and an acquisition rule to pick the next team to evaluate; **UCB**-style regret bounds apply. Natural when tier-2/3 evaluations are costly.
- **Game-theoretic (the principled version).** The meta is not fixed while you optimise — if you find a team that beats the field, the field adapts. The right object is a **Nash of the team-selection metagame**, approximated by **double-oracle / PSRO** (Policy-Space Response Oracles): alternately compute a best-response team to the current population and add it, growing toward equilibrium. This is exactly the "iterate on itself" the flywheel calls for, and it is the game-theoretic baseline VGC-Bench implements.

Coarse-to-fine ties the tiers together: tier 1 ranks thousands of candidate teams, tier 2 re-ranks the top few hundred, tier 3 vets the handful of finalists. No expensive evaluation is ever spent on an obviously bad team.

9. The self-improving flywheel, formalised

The five ABRA stages form a fixed-point iteration. Let `D_k` be the meta distribution at round `k`, `M_k` the model fit on data up to `k`, and `Π_k` the optimised team(s). Then:

```
M_k   = Fit(Data_k)                       # learn the model from logged
games
Π_k   = Optimise(M_k, D_k)                # best team(s) vs the measured
meta
Data_{k+1} = Data_k ∪ Play(Π_k)           # play; ingest every opponent
faced
D_{k+1}    = Meta(Data_{k+1})             # re-measure the meta
```

This is **expert iteration** — the AlphaZero pattern — with the environment supplied by the live ladder rather than self-play. Its appeal is compounding: each turn of the crank adds data where the current model is *most uncertain* (the games you actually play), which is the most valuable data to acquire. Its dangers are equally real and must be designed against: **feedback loops** (optimising against a meta you also perturb), **distributional collapse** (a model that only ever sees its own preferred games forgets the rest of the ladder — mitigated by continuing to ingest the *whole* public ladder, not only your games), and **non-stationarity** (the meta drifts under you; models must be re-fit, not frozen). The flywheel is powerful *because* it is a control loop, and it must be treated with a control loop's caution.

10. Related work (grounding, 2024–2025)

- **PokéChamp** (Karten, Nguyen, Jin; ICML 2025 spotlight). LLM-augmented **minimax** tree search: the LLM supplies action sampling, opponent modelling, and value estimation inside a two-player game tree; no extra training; Elo 1300–1500 (top 10–30% of humans); ships the largest real-player dataset (3M+ games, 500k+ high-Elo). Directly

validates tier-3-by-search (§5.3).

- **Metamon** (UT Austin RPL; RLC 2025). **Offline RL** with transformers on 5M+ trajectories reconstructed from human Showdown replays plus 20M+ self-play — the offline, learn-from-logs setting that is ABRA's exactly. Validates tier-3-by-learned-policy (§5.4).
- **VGC-Bench** (Angliss et al., 2025). A benchmark for *VGC specifically*, with the combinatorial team-strategy generalisation problem front and centre, and baselines spanning **behaviour cloning, multi-agent RL (PPO), game-theoretic (double-oracle), LLM, and heuristics** — the method menu of §5 and §7. Also extends poke-env with PettingZoo/VGC support.
- **PokéAgent Challenge** (NeurIPS 2025). Establishes Pokémon as a standard testbed for sequential decision-making under uncertainty, with Metamon and PokéChamp as baselines.
- **Classical foundations.** Bradley–Terry (1952) and Elo for paired comparisons (§4.2); CFR / Deep CFR for imperfect-information equilibria (§2); Dreamer and MuZero for learned-model planning (§5); CQL, IQL, Decision Transformer for offline RL (§5.4); PSRO / double-oracle for empirical-game equilibria (§7).

The takeaway: none of the three tiers is speculative. Each corresponds to a demonstrated method in the current literature. What ABRA contributes is not a new algorithm but the **data pipeline and the flywheel** that make these methods self-sustaining on a live, closed-engine format.

11. Evaluation protocol

Every tier is scored on **held-out games** — a temporal split (train on older games, test on newer), never random, to respect meta drift and avoid leakage.

- **Outcome model (tier 1).** Test log-likelihood, **Brier score**, calibration reliability diagram, and AUC. Report against two baselines: the constant 50% predictor, and the Bradley–Terry main-effects-only

model. A model that cannot beat "the higher-usage team wins" is not yet useful.

- **Dynamics model (tier 3).** Per-step next-state log-likelihood and long-horizon rollout divergence (does a 10-turn rollout stay near reality?). Policy quality by **off-policy evaluation** and, ultimately, by **live ladder Elo**, the only unarguable metric.
- **The honest ceiling.** Pokémon has irreducible variance: imperfect information plus damage rolls, accuracy, and speed ties mean even a perfect model cannot predict single games with high accuracy — which is why the strongest published agents plateau around Elo 1300–1500 rather than dominating. A prior internal analysis on a coverage heuristic found game-level correlation ≈ 0; that finding stands as the caution. We therefore state success as **calibrated aggregate win-rate improvement of optimised teams over a baseline**, measured over many games, not per-game oracular accuracy.

12. Feasibility — can we actually build this?

Yes, tier by tier, with honesty about each. **As of v1.4, tiers 1 and 2 and the DITTO outer loop are built and measured; tier 3 remains an honest research effort.**

- **Tier 1 (JOLTEON) is built (v2).** A Bradley–Terry logistic model over per-species strengths, now with **rarity-aware L2 shrinkage** (a species is trusted in proportion to its sample size — seen 25×, its rating is pulled toward neutral; seen 1000×, it is trusted) plus speed-edge and firepower-edge features from the dynamics model. On a fresh 4,999-game pull (temporal split, humans only): **~55% held-out accuracy vs ~49% for a coin flip**, Brier ~0.25 (calibrated). The number sits 55–57% depending on the exact games/split — modest, exactly as this domain's variance predicts, but genuinely above chance from team composition alone. Honest ablation: the dynamics features *tie* species-only at this scale (firepower earns weight +0.30; speed-edge is noise), and the rarity shrinkage costs no accuracy while removing the overfit

an optimiser could exploit. Antisymmetry and calibration are pinned by `tests/test-jolteon.py`.

- **Tier 2 (MEDICHAM) is built.** A Monte-Carlo rollout over CHOMP's exact damage engine with a heuristic (behaviour-cloned-later) move policy, sampling damage rolls. Sanity holds: rain core beats sun core **0.60**, mirror match **0.51**, ~1s per 300 playouts. v1 is sequential-singles (one active per side) — honest scope; true doubles turns are roadmap. Pinned by `tests/test-medicham.js`.
- **The DITTO outer loop is built** — and its most important output is a *negative* result stated plainly. Optimising a team to maximise JOLTEON's score **Goodharts the evaluator**: DITTO finds rare species JOLTEON overrates and stacks them, producing a team JOLTEON scores at **90%** that MEDICHAM's grounded rollouts reveal is **~12%**. This is caught, not hidden: DITTO now (a) restricts to a proven-meta pool, (b) uses **usage-weighted threat coverage** so it guarantees an answer to high-bring threats (Basculegion) and ignores rare ones (Camerupt), (c) prints a **bias report** showing where rarity shrinkage suppresses a pick, and (d) vets every finalist with MEDICHAM. The JOLTEON-low / MEDICHAM-high disagreement is itself a signal — a genuine sleeper the data has not yet confirmed. This is the tiered design (§3) earning its keep.
- **The dynamics model is built** (`engine/dynamics.js`): the per-turn stream now yields observed speed order (186 species, with Choice-Scarf detection) and observed damage distributions (1,170 attacker|move pairs). This is the empirical grounding tier 3 will calibrate against, and it already feeds tier 1 and KADABRA.
- **Tier 3 (SLOWKING) is a research effort, scaffolded not trained.** A learned dynamics model, belief tracking, and equilibrium search (or offline-RL policy) is the frontier the cited papers occupy. It is *possible* — they prove it — but it needs real compute, careful offline-RL practice, and sustained work, judged against the tier-2 baseline before its cost is accepted. The interface is fixed (`engine/slowking.py`) and the data toward it is already collecting: **30,611 per-turn state**

transitions over 4,999 games, with raw logs archived so nothing must be re-pulled. Grey-box structure (§5.1) and a warm start from open Showdown data are the levers that make it tractable.

The disciplined path — ship tier 1, keep the flywheel turning, add tier 2 for finalist vetting, treat tier 3 as a research track whose bar is set by tier 2 — is now demonstrated end-to-end, not just proposed.

13. References

1. Karten, S., Nguyen, A. L., Jin, C. (2025). *PokéChamp: an Expert-level Minimax Language Agent.* ICML 2025 (spotlight). arXiv:2503.04094.
2. UT Austin RPL (2025). *Metamon: Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers.* RLC 2025. arXiv:2504.04395.
3. Angliss, C. et al. (2025). *VGC-Bench: A Benchmark for Generalizing Across Diverse Team Strategies in Competitive Pokémon.* arXiv:2506.10326.
4. *PokéAgent Challenge*, NeurIPS 2025.
5. Bradley, R. A., Terry, M. E. (1952). *Rank Analysis of Incomplete Block Designs.* Biometrika.
6. Zinkevich, M. et al. (2007). *Regret Minimization in Games with Incomplete Information (CFR).* NIPS.
7. Kumar, A. et al. (2020). *Conservative Q-Learning for Offline RL.* NeurIPS.
8. Kostrikov, I. et al. (2021). *Offline RL with Implicit Q-Learning (IQL).*
9. Chen, L. et al. (2021). *Decision Transformer: RL via Sequence Modeling.* NeurIPS.
10. Schrittwieser, J. et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero).* Nature.
11. Lanctot, M. et al. (2017). *A Unified Game-Theoretic Approach to Multiagent RL (PSRO).* NeurIPS.

12. Chen, S., Joachims, T. (2016). *Modeling Intransitivity in Matchup Outcomes (Blade–Chest).* ICML.
13. Brown, N. et al. (2020). *Combining Deep RL and Search for Imperfect-Information Games (ReBeL).* NeurIPS.
14. Schmid, M. et al. (2021/2023). *Player of Games* / *Student of Games:* unified search + learning across perfect- and imperfect-information games. Science (2023).
15. Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero).* ICML.
16. Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel (Gumbel MuZero).* ICLR.
17. Vinyals, O. et al. (2019). *Grandmaster level in StarCraft II (AlphaStar).* Nature; and *AlphaStar Unplugged* (2023), large-scale offline RL.
18. French, R. (1999). *Catastrophic Forgetting in Connectionist Networks.* Trends in Cognitive Sci.
19. Hooper, W. (2026). *CHOMP white paper, validation report, and referee report* — the Gen-9 damage pipeline, the pKO-over-16-rolls threat score, and the coverage-based bring-4, used here as JOLTEON's feature generator and MEDICHAM's dynamics. `Pokemon/CHOMP/docs/`.

Companion documents. ABRA white paper · Executive summary · Technical documentation

PART 7

KADABRA — Coach Product Spec

Key Analysis of Decisions, Advice & Better Replay Annotation

Version 0.1 (spec) · 2026-07-22 · Will Hooper

> *The interactive coaching front-end of the ABRA platform. ABRA collects and models the ladder;* ***KADABRA is the psychic that shows it back to you*** *— it walks you through your own game turn by turn, recreates each scene, tells you the better play, and answers your questions before you move on. Abra → Kadabra: the data platform's teaching evolution.*

---

1. What it is

A **chatbot coach** for a single Champions replay. You paste a replay link; KADABRA reviews the game *with* you, one decision at a time. It is not a wall-of-text report (that is the `coach.js` summary). It is a conversation that stops at each meaningful turn, **reconstructs the scene**, evaluates your play against the models, and waits for your questions before advancing.

It replaces "send Claude a link and have it watch the game" with a purpose-built, instant, repeatable tool — and every game you review is silently added to ABRA's database and folded into the model (the flywheel, from the coaching seat).

2. The core loop (per turn)

1. **Reconstruct the scene.** From the parsed log, KADABRA rebuilds the board at this turn: the two active Pokémon per side, HP bars, status, stat stages, weather/terrain/screens, and what is known of each team. It *shows* this, not just describes it — a rendered board state, so you see

the scenario the way it looked in-game.

2. **State what happened.** "You led Pelipper + Swampert into Gengar + Hydrapple; rain is up; Gengar mega'd and put Swampert to sleep."
3. **Evaluate the decision.** KADABRA compares the move you made against the model's recommended play (see §4), with the reason: the damage math (CHOMP), the threat read (ABRA usage), and — when available — the deeper line (SLOWKING).
4. **Answer your questions.** You can ask anything — "why not Protect?", "would Ice Beam have KO'd?", "what if they didn't switch?" — and KADABRA answers from the same models, running the damage calc or a rollout on demand.
5. **Advance on your say-so.** Only when you're ready does it move to the next decision point. You control the pace; it never dumps the whole game at once.

3. Scene reconstruction

The parser (`engine/durable-ingest.js` / `engine/coach.js`) already reconstructs the full state progression from the protocol log — switches, moves, damage, faints, field changes. KADABRA renders that state at any turn as a simple board: two active slots per side with sprite, HP %, status chip, and a field banner (weather/terrain/Trick Room/Tailwind). Because the whole trajectory is reconstructable, the player can also scrub — "show me turn 5" — and KADABRA redraws that scene.

4. The advice — where the models plug in

"The correct play might have been…" is answered at the fidelity available:

- **Now (v1).** CHOMP's exact damage engine gives the KO math for every option (pKO across 16 rolls, speed order), and ABRA's usage model gives the opponent read (their likely set/lead). Together these already justify most coaching calls — "Wave Crash OHKOs their Garchomp in rain; you clicked Protect and gave them a free turn."

- **Later (roadmap). SLOWKING** (the learned belief-search solver, simulator white paper §5) supplies the deep line — the equilibrium-aware best move accounting for what the opponent might do — and **MEDICHAM** rollouts answer "what if" branches. KADABRA's advice quality rises as those models come online, with no change to the interface.

KADABRA is a **consumer** of the models (MEDICHAM, JOLTEON, SLOWKING) and of CHOMP, not a model itself.

5. The background job — the flywheel from the coaching seat

Every replay KADABRA reviews is passed through the ABRA extractor and appended to `data/games.ladder.jsonl` (dedup by id). So a coaching session is also a data-collection event: the opponent's team, sets revealed, and result enter the store and improve the models — exactly the feed-back stage of the flywheel, triggered by the act of reviewing your own game.

6. Interface

A single conversational panel: the reconstructed scene on top, the coach's message below, a text box for your questions, and a "next decision →" control. Optionally a side rail with the turn timeline so you can jump around. Web-based, so it can live alongside the ABRA dashboard.

7. Honest status and scope

- **Buildable now:** the parser, scene reconstruction, CHOMP-based advice, the Q&A loop over damage math, and the background ingest. This is a real v1.
- **Roadmap:** the *depth* of "the correct play" scales with SLOWKING and JOLTEON; until then advice is grounded in exact damage + usage, which is honest and already strong, but single-ply.

- **Own project?** KADABRA is a distinct product (its own app and, when built, its own repository and full doc set), connected to ABRA (data/models) and CHOMP (damage). It is specced here inside ABRA because it is the platform's front-end; it graduates to its own project folder when we build it.

8. The named cast (so the pipeline is legible)

Name	Pokémon	Role
ABRA	Abra	collects live ladder games, stores + models them
JOLTEON	Jolteon (fastest)	instant team-vs-team win probability
MEDICHAM	Medicham (medium)	quick physics rollouts
SLOWKING	Slowking (slow, wise)	deep learned belief-search game solver
DITTO	Ditto	team optimiser vs the meta
CHOMP	Garchomp	the bring-4 / lead-2 engine (exact damage)
KADABRA	Kadabra	the coach — reviews your games and teaches

Related. ABRA white paper · Simulator white paper · Executive summary